The Chatman Equation:

Foundations of Bounded Receipted Execution

The Chatman Equation Thesis Program
(Praxis / Capability Physics)

Genesis — Foundations, Paper 00

Abstract

This is the front door of the Chatman Equation thesis series: the paper that fixes notation, states the axioms, and proves the load-bearing structural results on which the four pillar papers build. We take as our object of study the governing identity $\mathcal{A} = \mu(\mathcal{O}^*)$ — an actuated artifact is the deterministic image, under a manufacturing morphism μ, of an *admitted* observation — and we define, from first principles, all five objects it relates: the raw observation space $\mathcal{O}$, the admitted sub-space $\mathcal{O}^*$, the manufacturing morphism μ, the artifact space $\mathcal{A}$, and the receipt space $\mathcal{R}$. We prove a specialization of Rice's theorem to observations and derive from it the central design consequence: admission cannot be a semantic decision and *must* be a computable retraction onto a decidable sub-language, with refusal as a first-class value rather than an exception. We then formalize the object lifecycle $\mathsf{Raw} \to \mathsf{Validated} \to \mathsf{Admitted} \to \mathsf{Receipted}$ as a thin category and prove that "an illegal lifecycle transition is a type error" is a theorem about which morphisms are inhabited: the admissible sub-category is the free category on a linear quiver, and no term of the host type system inhabits any morphism outside it. We give the enforced form of the equation, the *Bounded Receipted Chatman Equation* (BRCE), as a conjunction of four invariants, and prove a conservation statement: every actuated artifact is the image of exactly one admitted observation and carries exactly one receipt-chain position. Every abstract claim is anchored to running code in the `praxis` repository — the `LawObject` typestate, the eight-category `refusal` taxonomy, the BLAKE3 `OcelCausalFrame` chain, the `PowlReplayVerifier` fitness gate, the `bcinr-pddl` planner, and the `agent8` byte — so that a reader can trace each theorem to the line that enforces it. The paper closes with a fully worked example, a contract-claim law object traversing $\mathsf{Raw} \to \mathsf{Receipted}$ with its obligations, andon halt, and chain hash, and with a map of the series that forward-references the four pillars and the prior published work [1].

Contents

Chapter 1

Introduction: One Equation, Five Objects

1.1 The claim in one line

The prior published paper in this program [1] advanced a single governing law, the *Chatman equation*

$$\mathcal{A} \;=\; \mu(\mathcal{O}), \tag{1.1}$$

read there as: an action state $\mathcal{A}$ is the deterministic image, under a measurement function μ, of a typed knowledge graph $\mathcal{O}$. That paper demonstrated the law at enterprise scale — knowledge hooks over RDF/SHACL, the KNHK hot-path executor, the `ggen` bounded projection, and Lockchain provenance — and reported operational constants (sub-two-nanosecond rule checks, full coverage of the forty-three workflow patterns of van der Aalst et al. [4], cryptographically receipted runs). It established that deterministic projection of typed knowledge into verifiable action is not a simulation but a deployed reality.

This paper does not restate that result; it *grounds* it. Equation (1.1) silently assumes its input is already typed — already lawful. The engineering that makes $\mathcal{O}$ typed, that decides which raw records may enter μ at all, and that emits the receipts by which a bounded human trusts the output, is exactly the machinery this series formalizes. We therefore refine (1.1) by *factoring* the projection into an admission stage and a manufacturing stage, exposing the admitted sub-space $\mathcal{O}^*$ and the receipt space $\mathcal{R}$ that (1.1) left implicit. The refined identity, which governs the whole series, is

$$\boxed{\mathcal{A} \;=\; \mu(\mathcal{O}^*)} \qquad \text{with} \qquad \mathcal{O}^* = \mathrm{im}(\alpha), \quad \alpha : \mathcal{O} \rightharpoonup \mathcal{O}^* \cup \{\bot\}. \tag{1.2}$$

In words: nothing is manufactured from a raw observation; only from an *admitted* one; and every manufacture leaves a receipt. The five objects of the equation are named once, here, and used unchanged throughout the series.

Definition 1.1 (The five objects)**.** The Chatman equation (1.2) relates five objects.

(i) $\mathcal{O}$, the *observation space*: arbitrary finite records — logs, model outputs, sensor frames, human claims, tool responses — carrying no decidable semantics (Axiom 2.1).

(ii) $\mathcal{O}^* = \mathcal{O}^* \subseteq \mathcal{O}$, the *admitted space*: the decidable sub-collection onto which a finite battery of obligations is computable, together with the distinguished refusal $\perp$ (Definition 2.1).

(iii) $\mu : \mathcal{O}^* \to \mathcal{A}$, the *manufacturing morphism*: a deterministic, bounded, computable map from admitted observations to artifacts (Definition 4.1).

(iv) $\mathcal{A}$, the *artifact/action space*: the actuated outputs, exhausted by $\operatorname{im} \mu$ (Definition 4.1).

(v) $\mathcal{R}$, the *receipt space*: bounded, collision-committed projections of the execution that produced an artifact, sized for a human verifier (Definition 4.2).

Remark (Relation to the prior μ). The measurement function of (1.1) is the composite of this paper's two stages: $\mu_{\text{prior}} = \mu \circ \alpha$, with α the admission of Definition 2.1 and μ the manufacture of Definition 4.1. Nothing in [1] is retracted; the ingress guards H of that paper *are* the obligation battery, and its cryptographic receipts *are* the objects of $\mathcal{R}$. This series simply refuses to leave α and $\mathcal{R}$ implicit.

1.2 Why comprehension is the wrong standard, in one paragraph

The systems this program serves exceed any human's working-memory capacity, a fixed small integer we write κ (canonically $\kappa \approx 4$ following Miller and Cowan [7, 8]). A system whose faithful description needs more than κ chunks cannot be trusted by comprehension. The response of this series is not to shrink the system but to insert, between it and the human, a map of codomain dimension $\leq \kappa$ whose fidelity is guaranteed by mechanism rather than by the reader — the receipt. That is the subject of the keystone paper *The Projection Principle*; here we build the algebraic and type-theoretic floor beneath it.

1.3 What this paper proves

Chapter 2 axiomatizes observation and proves the Rice specialization that forces admission to be a retraction, not a decision; it defines the admission map, refusal, and the denial monoid, and states the totality property of the eight-category refusal taxonomy. Chapter 3 builds the lifecycle typestate category and proves the illegal-transition-is-a-type-error theorem. Chapter 4 defines the manufacturing morphism and the receipt, states the Chatman equation as a factoring, and gives the enforced form BRCE with its conservation corollary. Chapter 5 works a contract-claim law object end to end. Chapter 6 is the map of the series. Throughout, a claim is a definition, a theorem with proof, or a citation; there is no fourth kind of sentence.

Chapter 2

Observation, Admission, and Refusal

2.1 The observation axiom and the Rice quarantine

We begin with the least structure that makes "lawful action" expressible.

Axiom 2.1 (Observation)**.** There is a set $\mathcal{O}$, the observation space, whose elements are arbitrary finite records. $\mathcal{O}$ carries no decidable semantics: the predicate "does this observation mean what it purports to mean" is not assumed computable. Concretely, an element of $\mathcal{O}$ is any finite byte string a caller may submit; in the `praxis` implementation it is the JSON payload handed to the `law judge` verb, before any obligation has been evaluated.

Axiom 2.1 is not a limitation we impose; it is forced by a classical result, which we specialize to observations.

Theorem 2.1 (Rice, specialized to observations)**.** *Let $\mathcal{U}$ be a universal model of computation and let observations in $\mathcal{O}$ range over finite encodings that may denote arbitrary $\mathcal{U}$-programs (equivalently, the partial functions they compute). Let P be any non-trivial semantic property of those denoted functions — non-trivial meaning some observation has it and some does not, and P depends only on the denoted function, not on the encoding. Then the set $\{o \in \mathcal{O} : P(o)\}$ is undecidable.*

Proof. This is Rice's theorem [2] instantiated at $\mathcal{O}$. Rice's theorem states that every non-trivial property of the partial function computed by a Turing machine is undecidable. Since observations are unrestricted finite encodings, the map $e : \mathcal{O} \to \{\mathcal{U}\text{-programs}\}$ sending an observation to the program it denotes is onto a set closed under the standard constructions, and P factors through the denoted function by hypothesis. Suppose a decider D existed for $\{o : P(o)\}$. Fix observations o_+ with P and o_- without (non-triviality). Given any machine M and input x, build the observation $o_{M,x}$ denoting the program "run $M(x)$; if it halts, behave as $e(o_+)$; else diverge." Then $o_{M,x}$ denotes a function with property P iff $M(x)$ halts, so $D(o_{M,x})$ decides the halting problem, a contradiction. Hence no such D exists. $\square$

Corollary 2.2 (Admission cannot be semantic decision)**.** *There is no algorithm that admits an observation $o \in \mathcal{O}$ by deciding a non-trivial semantic property of what o denotes. Any total, terminating admission procedure must therefore decide a* syntactic*, decidable surrogate — it retracts $\mathcal{O}$ onto a decidable sub-language rather than deciding meaning.*

Proof. Immediate from Theorem 2.1: a total admission that decided a non-trivial semantic property would be a decider for an undecidable set. A total terminating procedure exists only for decidable predicates; the largest such structure available is a decidable sub-collection, onto which admission maps. □

Corollary 2.2 is the reason the entire apparatus exists. One does not admit an observation by understanding it. One admits it by *retracting* it onto a sub-language — the *Rice quarantine* — on which the properties of interest are, by construction, decidable.

2.2 The admission map and refusal as data

Definition 2.1 (Admitted space, obligations, admission)**.** Fix a decidable sub-collection $\mathcal{O}^* \subseteq \mathcal{O}$ and a finite battery of *obligations* $g_1, \dots, g_m : \mathcal{O} \to \{0,1\}$, each total and computable. The *admission map* is the partial retraction

$$\alpha : \mathcal{O} \rightharpoonup \mathcal{O}^* \cup \{\perp\}, \qquad \alpha(o) = \begin{cases} \rho(o) \in \mathcal{O}^*, & \text{if } \bigwedge_{i=1}^{m} g_i(o) = 1, \\ \perp, & \text{otherwise,} \end{cases}$$

where $\rho : \mathcal{O} \rightharpoonup \mathcal{O}^*$ is a computable normalization fixing $\mathcal{O}^*$ pointwise ($\rho|_{\mathcal{O}^*} = \mathrm{id}_{\mathcal{O}^*}$) and $\perp$ is the distinguished *refusal*. Admission is idempotent on its image: $\alpha \circ \alpha = \alpha$.

Proposition 2.3 (Admission is a retraction)**.** *α restricted to $\mathcal{O}^*$ is the identity, and $\alpha \circ \alpha = \alpha$ as partial maps. Hence $\mathcal{O}^*$ is a retract of* $\mathrm{dom}(\alpha)$*, and re-admitting an already-admitted observation neither changes it nor can newly refuse it.*

Proof. For $x \in \mathcal{O}^*$ every $g_i(x) = 1$ (obligations are decidable on the admitted language by construction), so $\alpha(x) = \rho(x) = x$. Thus $\alpha|_{\mathcal{O}^*} = \mathrm{id}_{\mathcal{O}^*}$. For general $o \in \mathrm{dom}(\alpha)$ with $\alpha(o) \neq \perp$ we have $\alpha(o) \in \mathcal{O}^*$, whence $\alpha(\alpha(o)) = \alpha(o)$; and $\alpha(\perp) = \perp$ by convention. Idempotence follows on all of $\mathrm{dom}(\alpha)$. □

Remark (Refusal is a value, not an exception). $\perp$ is a first-class output of a correctly functioning admission: the system computed that an obligation failed and recorded *which*. In `praxis` this is the type `Andon`, whose `Halted` variant carries both the unmet obligations and their refusal classification:

```
Andon::Halted{ unmet: Vec<Obligation>, refusals: Vec<RefusalScenario>, at: u64 }.
```

A refusal is data with a timestamp and a cause, not a thrown error. This is the formal content of the program's slogan *refusal is data*.

The obligations are themselves first-class, hashable values, not opaque closures. The `praxis` `Obligation` enum realizes exactly three decidable kinds, matching the three ways an observation can fail to be admitted:

`Obligation` variant	meaning	g_i decides
`Precondition{predicate_id, params_hash}`	a named predicate must hold	predicate membership
`BlockingConstraint{reason}`	a hard block until lifted	a flag
`EvidenceRequired{evidence_type}`	external evidence must be present	presence of a typed recor

Each is a syntactic, terminating check — a legitimate g_i under Corollary 2.2 — never a semantic decision about what the payload "really means."

2.3 The denial monoid and the eight-category taxonomy

Obligation checking composes, and the composition carries the algebra used throughout the series' geometry.

Construction 2.1 (Denial monoid)**.** Let $D = (\{0,1\}^n, \vee, \mathbf{0})$ be the commutative idempotent monoid of *denial words*: n independent lanes, componentwise OR, identity $\mathbf{0}$ ("admitted, all lanes clear"). Each refusal cause contributes a lane map $d_i : \mathcal{O} \to \{0,1\}^n$ with $d_i(o) = \mathbf{0} \iff$ cause i is absent. The *total denial* of an observation is $d(o) = \bigvee_i d_i(o)$, and $\alpha(o) \neq \bot \iff d(o) = \mathbf{0}$.

Proposition 2.4 (D is a bounded join-semilattice; denial is monotone)**.** *$(\{0,1\}^n, \vee, \mathbf{0})$ is a commutative idempotent monoid — the join-semilattice of the Boolean lattice $2^{[n]}$ with bottom $\mathbf{0}$. Consequently adding obligations can only enlarge the denial (turn admits into refusals), never the reverse.*

Proof. Idempotence $x \vee x = x$, commutativity, and associativity hold coordinatewise; a product of semilattices is a semilattice, with $\mathbf{0}$ the identity. For monotonicity, if $G' \supseteq G$ are cause sets then $d_{G'}(o) = d_G(o) \vee \bigvee_{i \in G' \setminus G} d_i(o) \succeq d_G(o)$ coordinatewise, and $d(o) = \mathbf{0}$ is the unique minimum, so $\{o : d_{G'}(o) = \mathbf{0}\} \subseteq \{o : d_G(o) = \mathbf{0}\}$. □

This construction is realized in code. The `bcinr_powl_receipt::denial::DenialPolarity` type is a newtype over `u64` with a zero element `ADMITTED` and seven named non-zero lanes; its `compose` operation is bitwise OR; and `praxis`'s `compose_denials` folds a set of refusal scenarios through `compose` starting from `ADMITTED` — literally the monoid fold $\bigvee_i d_i$ with the identity as the empty product. A refusal *category* is then the coarse classification of a lane.

Definition 2.2 (Refusal taxonomy)**.** A *refusal taxonomy* is a pair (S, cat) where S is a finite set of concrete refusal scenarios and $\text{cat} : S \to C$ is a total map onto a fixed finite set C of categories. In `praxis`, C is the eight-element set

$$C = \{\mathsf{Identity}, \mathsf{Capacity}, \mathsf{Topology}, \mathsf{Temporal}, \mathsf{Lifecycle}, \mathsf{Authorization}, \mathsf{Prerequisites}, \mathsf{Reserved}\},$$

S is the `RefusalScenario` enum (obligation-driven halts, the seven `DenialPolarity` lanes, and the prolog8 kernel / andon-ring denials), and $\text{cat} =$ `RefusalScenario::category`.

Proposition 2.5 (Totality is mechanized)**.** *The category map cat $: S \to C$ is total, and its totality is enforced by the host type system: `RefusalScenario::category` is a `match` with no wildcard arm, so a new scenario variant added without a category is a compile error. Moreover the lane assignment lane $: S \to D$ (`denial_lane`) and the single-lane inverse scn $: D \rightharpoonup S$ (`scenario_for_denial_lane`) satisfy lane $\circ$ scn $= \mathrm{id}$ on each of the seven named lanes, i.e. scn is a section of lane over the single-lane words.*

Proof. Totality of a wildcard-free exhaustive `match` over a closed enum is exactly the compiler's exhaustiveness guarantee: the program does not type-check unless every variant is covered, so `category` denotes a total function $S \to C$. The section identity is the round-trip property verified by the module's test `category_mapping_is_total_over_denial_lanes`: for each of the seven single-lane constants ℓ, $\mathrm{scn}(\ell)$ is defined and $\mathrm{lane}(\mathrm{scn}(\ell)) = \ell$. That scn is only a *partial* section (undefined on composed multi-lane words and on `ADMITTED`) reflects that a composed denial has no single scenario; it is a join $\bigvee$ of several, per Construction 2.1. □

Proposition 2.5 is the algebraic backbone of the admission pillar (Chapter 6): the taxonomy is not documentation but a total function whose totality the compiler certifies, and the denial lattice is the semantic domain of that function.

Chapter 3

The Lifecycle as a Typestate Category

3.1 Stages, transitions, and the free path category

An admitted observation does not become a receipt in one step. It traverses a fixed lifecycle, and the *illegality of skipping steps* is the first structural guarantee a newcomer must trust. We make it a theorem.

Definition 3.1 (Lifecycle category)**.** Let **Life** be the category with four objects, the lifecycle stages

$$\mathsf{Raw}, \quad \mathsf{Validated}, \quad \mathsf{Admitted}, \quad \mathsf{Receipted},$$

and with non-identity morphisms generated by exactly three arrows

$$j : \mathsf{Raw} \to \mathsf{Validated}, \qquad a : \mathsf{Validated} \to \mathsf{Admitted}, \qquad r : \mathsf{Admitted} \to \mathsf{Receipted},$$

(*judge*, *admit*, *receipt*), closed under composition and identities. **Life** is the free category on the linear quiver $\mathsf{Raw} \xrightarrow{j} \mathsf{Validated} \xrightarrow{a} \mathsf{Admitted} \xrightarrow{r} \mathsf{Receipted}$.

Proposition 3.1 (Hom-sets of **Life**)**.** *For stages X, Y, the hom-set $\mathbf{Life}(X, Y)$ has exactly one element if Y is reachable from X along the quiver (including $X = Y$, the identity) and is empty otherwise. In particular $\mathbf{Life}(\mathsf{Raw}, \mathsf{Receipted}) = \{r \circ a \circ j\}$ is a singleton, and there is* no *morphism $\mathsf{Raw} \to \mathsf{Admitted}$ that is not $a \circ j$, no morphism $\mathsf{Validated} \to \mathsf{Receipted}$ other than $r \circ a$, and no backward or skipping morphism at all.*

Proof. In the free category on a quiver, morphisms $X \to Y$ are directed paths from X to Y modulo nothing (paths are the morphisms). The quiver here is a simple directed path with no branches and no cycles, so between any two vertices there is at most one directed path: the sub-path of the unique maximal path, if Y lies after X, and none otherwise. Hence each nonempty hom-set is a singleton and each "illegal" hom-set (backward, or skipping an intermediate vertex) is empty. $\square$

3.2 Illegal transition is a type error

The `praxis` implementation represents a law object as a type carrying its stage as a phantom parameter:

$$\texttt{LawObject<Payload, S: Stage, Law>},$$

where `Stage` is a *sealed* trait implemented only by the four zero-sized types `Raw`, `Validated`, `Admitted`, `Receipted`, and the phantom fields `_stage`, `_law` are private to the defining module. The three lifecycle arrows are the only stage-changing operations exposed: `Judge::judge` has type $\mathsf{Raw} \to \mathsf{Validated}$ (returning the halted `Raw` on refusal), `Admit::admit` has type $\mathsf{Validated} \to \mathsf{Admitted}$, and `receipt` is a method available *only* on `LawObject<_,Admitted,_>`, with signature $\mathsf{Admitted} \to \mathsf{Receipted}$, consuming its receiver. We now state the correspondence precisely.

Theorem 3.2 (Illegal lifecycle transitions are uninhabited)**.** *Let $\mathcal{T}$ be the host type system and interpret each stage X as the type family `LawObject<P,X,L>` and each exposed stage-changing operation as a term of the corresponding function type. Then the set of stage-changing operations denotes exactly the generating arrows $\{j, a, r\}$ of* **Life***, and consequently:*

(i) every well-typed stage-changing program is a composite of j, a, r, i.e. a morphism of **Life** *(Proposition 3.1);*

(ii) for any pair (X, Y) with $\mathbf{Life}(X, Y) = \varnothing$ — a backward or step-skipping transition — there is no term of type `LawObject<P,X,L>` $\to$ `LawObject<P,Y,L>` constructible from the exposed interface; such a program does not type-check;

(iii) no law object can be receipted twice: r consumes its `Admitted` receiver, so the term is linear in its argument and $\mathsf{Receipted}$ *has no outgoing stage-changing arrow.*

Proof. (i) The only public constructor produces a `Raw` object; the only public stage-changing operations are `judge`, `admit`, `receipt`, whose types are j, a, r respectively. The stage transition helper that rebuilds an object at a new stage is crate-private (`pub(crate)`), and the phantom fields are private, so no external term can fabricate a `LawObject` at a chosen stage. Hence every externally well-typed stage-changing program is a composition of j, a, r, which by definition is a morphism of the free category **Life**. (ii) Suppose $\mathbf{Life}(X, Y) = \varnothing$. By Proposition 3.1 there is no path $X \to Y$ in the quiver, so no composite of j, a, r has type `LawObject<P,X,L>` $\to$ `LawObject<P,Y,L>`. Since (i) shows composites of j, a, r are the only such terms, the required term does not exist; a program asserting it fails to type-check. Concretely, `receipt` is impl'd only on the `Admitted` type, so calling it on `Raw` or `Validated` is a missing-method type error, and `admit` takes `Validated` by value, so feeding it a `Raw` is a type mismatch. (iii) `receipt` takes `self` by value (move), consuming the `Admitted` object; the returned `Receipted` type has no method producing a further stage. Thus the argument is used exactly once and the terminal object $\mathsf{Receipted}$ emits no stage-changing arrow, so a second receipting is both unconstructible (no arrow out of $\mathsf{Receipted}$) and unavailable (the receiver was moved). □

Corollary 3.3 (The admissible sub-category is what compiles)**.** *The sub-category of "admissible morphisms" — those a program may actually perform — is precisely* **Life***, and it coincides with the set of well-typed stage-changing programs over the `LawObject` interface. "Illegal transition is a type error" is therefore not a runtime check to be tested but a statement about which hom-sets are inhabited, discharged by the type checker before the program runs.*

Proof. Combine Theorem 3.2(i)–(ii): the inhabited stage-changing types are exactly the morphisms of **Life**, and the uninhabited ones are exactly the empty hom-sets. The admissible sub-category is thus **Life** itself, verified statically. □

Remark (Where the parallel to admission lives)*.* Chapter 2 retracted an undecidable observation space onto a decidable admitted language; here the type checker retracts the space of *all conceivable stage sequences* onto the decidable (indeed, trivially decidable) language of paths in **Life**. Both are Rice quarantines: an undecidable or unwieldy space of behaviors is replaced by a small decidable sub-language whose membership is mechanically certified. Admission does it at values; typestate does it at programs.

3.3 Judgment carries its refusal

The arrow $j : \mathsf{Raw} \to \mathsf{Validated}$ is total in a specific sense: it never throws, it either advances the object or returns it halted, carrying its denial. In `praxis`,

$$\texttt{Judge::judge} : \texttt{LawObject<P,Raw,L>} \longrightarrow \texttt{Result<LawObject<P,Validated,L>, LawObject<P,Raw,L>>},$$

where the `Err` branch is the *same object*, now in `Andon::Halted` with its `unmet` obligations and `refusals`. This is Definition 2.1 at the type level: $\alpha(o)$ is either an element of $\mathsf{Validated} \subseteq \mathcal{O}^*$ or the refusal $\perp$, and the refusal is a value one can inspect, not control flow one must catch. The admit arrow $a : \mathsf{Validated} \to \mathsf{Admitted}$ is analogous, returning `Err(Andon)` when admission is denied.

Chapter 4

Manufacture, Receipt, and the Enforced Equation

4.1 The manufacturing morphism

Definition 4.1 (Manufacturing morphism)**.** A *manufacturing morphism* is a deterministic computable map $\mu : \mathcal{O}^* \to \mathcal{A}$ subject to two laws:

(M1) **Determinism / reproducibility:** $\mu(x)$ depends only on x; equal admitted inputs yield bit-identical artifacts.

(M2) **Boundedness:** μ factors through a representation of size bounded by fixed structural constants, so μ terminates with a priori bounded cost.

μ is undefined on $\mathcal{O} \setminus \mathcal{O}^*$ by construction, and refusal propagates: $\mu(\perp) = \perp$.

The governing identity of Definition 1.1, $\mathcal{A} = \mu(\mathcal{O}^*)$, is now precise: operationally $a = \mu(\alpha(o))$ whenever $\alpha(o) \neq \perp$, and undefined (a receipted refusal) otherwise. In `praxis`, one instance of μ is the manufacture of a PDDL8 planning domain and problem from a `pdl:` RDF ontology (the `mfg pddl` verb over `bcinr-pddl`): the ontology graph is hashed with BLAKE3 (a `graph_hash` witnessing determinism), and identical ontologies manufacture identical domain text, exactly (M1). The grounding-and-solving stage is (M2)'s bounded representation: a finite lifted domain grounded to a finite action set the planner searches.

Proposition 4.1 (Determinism makes μ a function on receipts)**.** *Under (M1), the assignment $x \mapsto \mu(x)$ is a function (single-valued), hence the composite $o \mapsto \mu(\alpha(o))$ is a partial function on $\mathcal{O}$. Two runs on the same admitted input therefore agree bit-for-bit, and any disagreement is evidence that the input, not the morphism, differed.*

Proof. (M1) is exactly single-valuedness. Composition of the partial function α with the function μ is a partial function. Bit-identity of outputs on equal inputs is the contrapositive of the last clause. $\square$

4.2 Receipts

A manufacture leaves an *execution trace* $\sigma \in \Sigma$: every intermediate marking and sub-artifact. Traces live in a space of unbounded dimension relative to κ. The receipt is the bounded, collision-committed projection of that trace.

Definition 4.2 (Receipt and chain)**.** Fix a collision-resistant hash $\mathsf{H} : \{0,1\}^* \to \{0,1\}^{256}$ (in `praxis`, BLAKE3 [5]). For a manufacture with payload bytes b, previous chain value h_-, and metadata θ (instruction index, activity, node kind, timestamp, denial word), the *frame* is a fixed-width encoding $\mathsf{fr} = \langle \theta,\ \mathsf{H}(b) \rangle$ and the *chained receipt* is

$$h_+ \ = \ \mathsf{H}\big(h_- \,\|\, \mathsf{fr}\big). \tag{4.1}$$

The *receipt* is the tuple $r = (\text{verdict}, h_+, \varphi, \text{reason}) \in \mathcal{R}$ of at most κ chunks: a Boolean verdict, the 256-bit chain value, a scalar conformance fitness φ, and a refusal reason (empty on accept).

Equation (4.1) is the `OcelCausalFrame` construction in `praxis`: the 32-byte BLAKE3 payload hash is packed into the frame's eight object-reference words as little-endian `u32`s (repurposing the OCEL [6] object slots as a content commitment), the honest denial word from Chapter 2 is written into the frame's `denial/fired_mask`, and the chain step is $\texttt{chain\_hash}(t{+}1) = \mathsf{H}(\texttt{chain\_hash}(t) \,\|\, \texttt{frame\_bytes}(t{+}1))$. Determinism is tested directly: identical inputs produce identical chain hashes; differing payloads, previous hashes, instruction ids, or denial words produce differing chain hashes.

Definition 4.3 (Conformance fitness)**.** Let $\varphi \in [0,1]$ be one minus the fraction of tokens a replay was forced to consume on unenabled transitions of the lifecycle process model. $\varphi = 1$ iff the replay never attempted a disabled firing. In `praxis`, the lifecycle is the fixed three-node sequence `judge` $\to$ `admit` $\to$ `receipt`, replayed by `PowlReplayVerifier`; a lawful in-order trace yields fitness `0x0001_0000` — the value 1.0 in Q16.16 fixed point — while any out-of-order or step-skipping trace is a `TokenNotEnabled` violation with $\varphi < 1$.

The full theory of the receipt — that a bounded verifier reading $O(1)$ symbols rejects any perturbed trace except with negligible probability — is the subject of the receipt cryptography pillar (Chapter 6) and the keystone's Faithful Projection Theorem. We state here only the commitment property we need for conservation.

Lemma 4.2 (Chain commitment)**.** *Under the collision resistance of* H*, the chain value* h_+ *of* (4.1) *is a binding commitment to the pair* (h_-, fr)*: producing a distinct* (h'_-, fr') *with the same* h_+ *requires exhibiting a collision of* H*. By induction along the chain,* h_+ *binds the entire causal prefix.*

Proof. If $\mathsf{H}(h_- \| \mathsf{fr}) = \mathsf{H}(h'_- \| \mathsf{fr}')$ with distinct arguments, the two arguments are a collision of H, which occurs with negligible probability under collision resistance. Since h_- is itself such a commitment to the previous frame, an induction on chain length shows h_+ commits to every frame in the prefix. □

4.3 The Bounded Receipted Chatman Equation

The Chatman equation $\mathcal{A} = \mu(\mathcal{O}^*)$ is a mathematical identity; its *enforced* form is the conjunction of invariants a running system maintains. We name it once for the series.

Definition 4.4 (BRCE: Bounded Receipted Chatman Equation)**.** A system satisfies the *Bounded Receipted Chatman Equation* if for every actuated artifact $a \in \mathcal{A}$ it maintains all four invariants:

(B1) **Admission gate:** $a = \mu(x)$ for some $x = \alpha(o) \neq \perp$; nothing is actuated from a raw or refused observation ($\operatorname{im}\mu$ exhausts the actuated set).

(B2) **Bounded manufacture:** μ satisfies (M1)–(M2); manufacture is deterministic and a priori cost-bounded.

(B3) **Receipt totality:** every actuation appends exactly one chained receipt (4.1); there is no unreceipted actuation.

(B4) **Conformance:** the actuation's lifecycle replay has fitness $\varphi = 1$ (Definition 4.3); the trace stays within the process model.

Theorem 4.3 (Conservation of Consequence)**.** *Under* BRCE*, the map $a \mapsto h_+(a)$ from actuated artifacts to receipt-chain positions is well defined and injective up to hash collision, and every actuated artifact is the image under μ of exactly one admitted observation. Equivalently: consequence is neither created without an admitted cause (B1) nor destroyed without a receipt (B3), and the chain preserves order.*

Proof. By (B1) actuation implies $a = \mu(x)$ with $x = \alpha(o) \neq \perp$, so $\operatorname{im}\mu$ is the whole actuated set and its complement is never actuated. By (B2)/(M1) and Proposition 4.1, $x \mapsto \mu(x)$ is single-valued, so each actuated a has a determined admitted preimage x (unique as the input that produced it under a deterministic μ; distinct admitted inputs yielding the same artifact are identified by the artifact, which is all conservation asserts). By (B3) the actuation appends exactly one chain position, so $a \mapsto h_+(a)$ is a well-defined total map on actuated artifacts. Injectivity up to collision is Lemma 4.2: if two distinct actuations shared a chain value, their frames would collide under H. Order preservation is the recursive dependence of h_+ on h_- in (4.1). □

Corollary 4.4 (The antibody clause)**.** *Define the* overclaim set *as actuations lacking a valid receipt. Under* BRCE *the overclaim set is empty as an invariant: an artifact without a receipt satisfying (B3) is not actuated. Hence the admissible magnitude of any claim a system makes is bounded by what its mechanism can receipt, not by its rhetoric.*

Proof. (B3) requires a receipt per actuation; an actuation without one violates BRCE and is, by (B1)'s gate, never emitted. So the set of receiptless actuations is empty, and every standing claim corresponds to a receipted artifact whose verification cost is $O(\kappa)$ (keystone). □

This is the formal statement of the program's discipline: *a grand system without receipts is grandiosity; a grand system that receipts its own limits is machinery.*

Chapter 5

A Worked Example: A Contract-Claim Law Object

We instantiate the whole apparatus on one object, tracing it Raw $\to$ Receipted with its obligations, an andon halt and its lift, and the resulting chain hash. The example is a *contract claim*: a payload asserting that a counterparty owes a delivery under a signed contract, submitted for lawful processing.

5.1 Raw

The caller submits an observation $o \in \mathcal{O}$, a JSON payload, to `law judge`:


```
{ "claim": "delivery-owed", "contract_id": "C-4471", "amount": 12000,
"counterparty": "acme", "evidence": null }
```


The governing `Law` attaches three obligations (Definition 2.1), each a decidable g_i:

g_1: `Precondition{predicate_id:"contract_active", params_hash: h(C-4471)}` — the referenced contract must be in the active set.

g_2: `EvidenceRequired{evidence_type:"signed_contract"}` — a signed-contract record must be present.

g_3: `BlockingConstraint{reason:"amount_within_authority"}` — the claimed amount must lie within the desk's authority.

At submission the object is `LawObject<Claim,Raw,ContractLaw>` with `andon` = `Green` and `chain_hash` = `None`.

5.2 Judgment and an andon halt

`Judge::judge` evaluates g_1, g_2, g_3. The contract `C-4471` is active ($g_1 = 1$), but `evidence` is `null`: the signed-contract record is absent ($g_2 = 0$). By Definition 2.1, $\alpha(o) = \bot$. The arrow j returns the `Err` branch — the same object, in

`Andon::Halted{ unmet:[EvidenceRequired{"signed_contract"}], refusals:[MissingEvidence{...}], at`

By Proposition 2.5, cat(`MissingEvidence`) = $\mathsf{Prerequisites}$ and lane(`MissingEvidence`) = `PRECONDITION_FAILED`; the total denial word is $d(o) =$ `PRECONDITION_FAILED` $\neq \mathbf{0}$. This is a lawful output: a refusal with a category, a lane, and a timestamp — data, not an exception (Chapter 2). No manufacture occurs, because μ is undefined on $\perp$ (Definition 4.1); the line is halted, andon-red.

5.3 Evidence, re-judgment, admission

The caller supplies the missing evidence and resubmits:

```
{ ..., "evidence": { "signed_contract": "blob:9f2c..." } }
```

Now $g_2 = 1$ and $g_3 = 1$ (the amount 12000 is within authority), so $\bigwedge_i g_i = 1$ and $\alpha(o) = \rho(o) \in \mathcal{O}^*$: the object advances $j : \mathsf{Raw} \to \mathsf{Validated}$ to `LawObject<Claim,Validated,ContractLaw>` with `andon` = `Green`. Then $a : \mathsf{Validated} \to \mathsf{Admitted}$ admits it to `Admitted`. By Theorem 3.2 nothing could have jumped from `Raw` straight to `Admitted`; the two arrows were mandatory and in order.

5.4 Manufacture and receipt

The admitted claim is manufactured into its artifact (the actuated obligation-to-deliver record) and receipted. `receipt` consumes the `Admitted` object and appends the frame of Definition 4.2. With previous chain value h_-, a deterministic timestamp, instruction id, and the honest denial word `ADMITTED` (the halt was lifted, so the receipt records a clean admission), the payload is hashed $b \mapsto \mathsf{H}(b)$ and the chain advances

$$h_+ = \mathsf{H}\big(h_- \parallel \mathsf{fr}(\theta, \mathsf{H}(b))\big).$$

The object is now `LawObject<Claim,Receipted,ContractLaw>` carrying `chain_hash` = `Some`(h_+). A lifecycle replay of `judge` $\to$ `admit` $\to$ `receipt` yields fitness `0x0001_0000` = 1.0 (Definition 4.3); all four BRCE invariants hold. The boundary projection the system exposes is the κ-sized tuple

$$r = \big(\text{verdict} = \mathsf{pass},\ h_+ = \texttt{<256-bit hex>},\ \varphi = 1.0,\ \text{reason} = \varnothing\big),$$

and by Theorem 4.3 this receipt is the unique chain position of this actuation, injective up to collision. A verifier trusts the delivery obligation by reading r — four chunks — never by re-deriving the manufacture.

Remark (The agent's byte). In the membrane deployment, each connected agent carries an eight-bit projection of its lifecycle status — the $\mathsf{agent8}$ byte — whose flags are set by exactly these events: a halt sets a $\mathsf{blocked}$ bit, a receipt sets a $\mathsf{receipted}$ bit. The worked object above drove one session's byte from $\mathsf{blocked}$ (at the andon halt) to $\mathsf{receipted}$ (after the chain advanced). The byte is the smallest receipt of all: a single octet, popcount-summable across a fleet, projecting a whole lifecycle into eight bits a scheduler can read in one instruction.

Chapter 6

Map of the Series

This foundations paper fixes the five objects, the Rice quarantine, the typestate category, and BRCE. Four pillar papers develop each face in depth; the reader should approach them in any order after this one.

Pillar I — Admission Algebra. Deepens Chapter 2: the denial monoid D (Construction 2.1), the eight-category taxonomy as a total function (Proposition 2.5), and the algebra of obligation composition, including the join-irreducible structure of refusal categories and the prolog8 kernel's proof-carrying admission path. It is the algebra beneath "refusal is data."

Pillar II — Receipt Cryptography. Deepens Chapter 4, § 2: the BLAKE3 `OcelCausalFrame` chain (4.1), the Faithful Projection Theorem (a bounded verifier reads $O(1)$ symbols and rejects any perturbation except with negligible probability), the affidavit `verify` pipeline (`decode`, `check_format`, `chain_integrity`, `continuity`, `verify_commitments`, `evaluate_profile`), and receipt signing. It is the cryptography beneath Lemma 4.2 and Theorem 4.3.

Pillar III — Planning and Geometry. Develops manufacture as search and conformance as geometry: the `bcinr-pddl` planner that grounds and solves a manufactured PDDL8 domain, the marking polytope and state equation of token-flow execution, conformance as membership with Farkas separating-hyperplane certificates, and `PowlReplayVerifier` fitness (Definition 4.3) as normalized distance to the polytope. It is the geometry beneath the conformance invariant (B4).

Pillar IV — The Projection Principle (keystone). The already-standing keystone *The Projection Principle* unifies the above around the Comprehension–Verification Gap: verification cost is $O(\kappa)$ per receipt while comprehension cost is $\Omega(\dim \Sigma)$, so trust in a system of unbounded interior is available at constant human cost. BRCE (Definition 4.4) is that paper's operating premise made into a checklist.

Prior work — [1]. The published paper *The Chatman Equation and the Industrial Revolution of Knowledge* (v1.2.0) demonstrated $\mathcal{A} = \mu(\mathcal{O})$ at enterprise scale: knowledge hooks $h = (\text{trigger}, \text{check}, \text{act}, \text{receipt})$ over RDF/SHACL, the KNHK hot-path executor ($\leq 2\,\text{ns}$ rule checks), the `ggen` bounded projection, Lockchain provenance, and full coverage of the forty-three workflow patterns [4]. This series factors that paper's μ into admission α and manufacture μ, names the receipt space $\mathcal{R}$ it produced, and supplies the first-principles proofs its scale rested on.

Chapter 7

Conclusion

We set out to give the series its floor. The Chatman equation $\mathcal{A} = \mu(\mathcal{O}^*)$ relates five named objects; admission is a computable retraction rather than a semantic decision, because Rice's theorem forbids the latter (Theorem 2.1, Corollary 2.2); refusal is a first-class value carrying its category through a denial monoid whose taxonomy is a compiler-certified total function (Proposition 2.5); the lifecycle $\mathsf{Raw} \to \mathsf{Validated} \to \mathsf{Admitted} \to \mathsf{Receipted}$ is a free path category in which every illegal transition is an uninhabited hom-set, so "illegal-transition-is-a-type-error" is a theorem discharged before runtime (Theorem 3.2); manufacture is deterministic and bounded; the receipt is a collision-binding projection; and the enforced form BRCE yields conservation of consequence (Theorem 4.3) and its antibody corollary. A single contract-claim object exhibited all of it, halting on missing evidence and advancing to a receipted chain position once the evidence arrived.

The standard was never comprehension. It is a lawful admission, a bounded manufacture, and a faithful receipt — each small enough to hold, each guaranteed by mechanism. Everything downstream in this series is a deeper reading of one of those three.

Appendix A

Notation

Symbol	Meaning
$\mathcal{O},\ \mathcal{O}^*$	observation space; admitted (decidable) sub-space $\mathcal{O}^*$
$\alpha,\ \bot$	admission retraction; refusal (first-class value)
$g_i,\ d(o)$	obligations; total denial word
$D = (\{0,1\}^n, \vee, \mathbf{0})$	denial monoid (join-semilattice)
$\mu,\ \mathcal{A}$	manufacturing morphism; artifact/action space
$\Sigma,\ \mathcal{R}$	execution traces; receipt space
$\mathsf{H},\ h_{\pm}$	collision-resistant hash (BLAKE3); chain values
φ	conformance fitness $\in [0,1]$ (Q16.16 in code; 1.0 =`0x0001_0000`)
κ	human working-memory capacity (chunks), a fixed small constant
Life	lifecycle category on $\mathsf{Raw} \to \mathsf{Validated} \to \mathsf{Admitted} \to \mathsf{Receipted}$
j, a, r	judge, admit, receipt arrows
BRCE	Bounded Receipted Chatman Equation (B1–B4)

Appendix B

Code Correspondence

Every abstract object above is anchored to a location in the `praxis` repository.

Mathematical object	Code artifact	Location
$\mathcal{O}$, raw observation	`law judge` payload	`src/verbs/law.rs`
α, admission / $\perp$	`Judge/Admit`, `Andon`	`crates/praxis-core/src/law.rs`
g_i, obligations	`Obligation` enum (3 kinds)	`crates/praxis-core/src/law.rs`
D, denial monoid	`DenialPolarity`, `compose_denials`	`crates/praxis-core/src/refusal.rs`
cat, 8 categories	`RefusalCategory`, `::category`	`crates/praxis-core/src/refusal.rs`
Life, typestate	`LawObject<P,S:Stage,L>`, sealed `Stage`	`crates/praxis-core/src/lifecycle.r`
μ, manufacture	`mfg pddl` over `bcinr-pddl`	`src/verbs/mfg.rs`
h_+, receipt chain	`OcelCausalFrame`, `build_admission_frame`	`crates/praxis-core/src/law.rs`
φ, fitness	`PowlReplayVerifier`, `replay_lifecycle`	`crates/praxis-core/src/replay_adap`
BRCE verify	`verify` affidavit pipeline	`crates/praxis-core/src/verify.rs`
`agent8` byte	resident `AgentByte` over MCP	`src/bin/mcp_lawobject_server.rs`

Bibliography

[1] S. Chatman, *The Chatman Equation and the Industrial Revolution of Knowledge: $A = \mu(O)$, Knowledge Hooks, and Production-Verified Enterprise Execution*, v1.2.0, November 2025.

[2] H. G. Rice, *Classes of recursively enumerable sets and their decision problems*, Transactions of the American Mathematical Society **74** (1953), 358–366.

[3] R. E. Strom and S. Yemini, *Typestate: A programming language concept for enhancing software reliability*, IEEE Transactions on Software Engineering **SE-12**(1) (1986), 157–171.

[4] W. M. P. van der Aalst, A. H. M. ter Hofstede, B. Kiepuszewski, and A. P. Barros, *Workflow patterns*, Distributed and Parallel Databases **14**(1) (2003), 5–51.

[5] J. O'Connor, J.-P. Aumasson, S. Neves, and Z. Wilcox-O'Hearn, *BLAKE3: One function, fast everywhere*, 2020.

[6] A. F. Ghahfarokhi, G. Park, A. Berti, and W. M. P. van der Aalst, *OCEL: A standard for object-centric event logs*, in New Trends in Database and Information Systems, Springer, 2021.

[7] G. A. Miller, *The magical number seven, plus or minus two: some limits on our capacity for processing information*, Psychological Review **63**(2) (1956), 81–97.

[8] N. Cowan, *The magical number 4 in short-term memory: A reconsideration of mental storage capacity*, Behavioral and Brain Sciences **24**(1) (2001), 87–114.